ChatGPT

Synergy Testnet-Beta Cluster-Local DAG Sublayer Integration and DAG Security Enforcement Plan

Executive summary

Synergy's published PoSy materials define a clustered, deterministic-finality, BFT-style protocol with epoch structure, cluster rotation as a security primitive, and authenticated intra-/inter-cluster communications (including bridge relays). In particular, PoSy is explicitly designed for partial synchrony and cooperative validator clustering rather than "everyone participates in every round." (See PoSy overview and clustering rationale in `03-proof-of-synergy-consensus.md`.)

The **correct** way to add DAG architecture into this design is **not** to replace PoSy finality with a pure DAG-finality protocol. Instead, the best-fit architecture is a **cluster-local certified DAG layer** that acts as a high-throughput dissemination (and *optional* pre-ordering) plane **inside each validator cluster**, while **PoSy dual-quorum + QC finality remains the canonical ordering/finality mechanism**. This matches Synergy's own existing DAG integration guidance: "finality remains QC-based," "DAG is cluster-local," and "availability before ordering." (`Synergy_DAG_Validator_Cluster_Integration_Plan.docx` ¶6–22).

This design aligns with canonical DAG-BFT research: Narwhal/Tusk explicitly separates **transaction dissemination** from **transaction ordering**, using a DAG mempool to scale throughput and robustness, and then composes with an ordering layer. [1] Bullshark demonstrates "zero extra communication" ordering on top of a DAG and emphasizes practicality and performance properties, including security preserved under a "quantum adversary" model. [2]

From the **current Testnet-Beta code snapshot** (`src.zip`), the validator node presently uses a classic "pool → propose block" pipeline: P2P receives transactions into a shared `TX_POOL` (`src/p2p/networking.rs` L1241–1269) and the consensus engine periodically clones that pool, validates each tx, and proposes a block containing the validated set (`src/consensus/consensus_algorithm.rs` L315–414). Leader selection already uses an entropy beacon + synergy score weighting (`src/consensus/consensus_algorithm.rs` L358–367), and finality is implemented as a **dual quorum** mechanism with vote aggregation into a QC (`src/consensus/dual_quorum.rs` L49–198). However, some critical "canonicalization gaps" must be resolved **before** a DAG rollout can be safe and deterministic:

The spec's cluster sizing and rotation model differs across sources. The PoSy updated spec states clusters are "size 10–50" and describes epoch-based clustering with a typical target `C_target` of ~30 (`PoSy_updated.docx` ¶91, ¶257), while the Rust registry's cluster formation uses a hard-coded max cluster size of 5 (`src/validator.rs` L313–316). Additionally, consensus config defaults to `validator_cluster_size: 7` (`src/config/mod.rs` L176–193) while `ProofOfSynergy::new()` falls back to `cluster_size = 30` if config is absent (`src/consensus/consensus_algorithm.rs` L200–204). These mismatches must be unified as part of Phase "protocol freeze & parameter canonicalization."

Synergy's roadmap explicitly schedules "DAG Validator Cluster Architecture" during Testnet-Beta (May 2026) and positions DAG execution as part of the eventual full-stack mainnet design. (`Synergy_Network_Roadmap (1).docx`, table 4 row 2; table 7 row 3).

Synergy's security specification emphasizes that consensus-layer defenses must be **structural** (fail-closed), and highlights replay prevention and encrypted mempool / fair ordering as core protocol defenses (`Synergy_Network_Security_Implementation_Specification.docx` ¶22, ¶65–69, ¶105–106). In DAG systems, many vulnerabilities are "network-plane" (eclipse, spam, withholding), so enforcement must connect directly to: (a) deterministic, signed message formats; (b) strict membership (cluster-scoped allowlists); (c) evidence capture; and (d) concrete penalties via validator status, Synergy Score penalties, and governance emergency actions.

Assumptions (explicit, "no specific constraint" was provided):

Cluster sizes and topology are treated as configurable; this plan assumes you will choose a single canonical cluster sizing model and apply it uniformly (config + registry + rotation). Conflicting defaults are treated as pre-existing technical debt to fix (see above). The uploaded `src.zip` contains the validator client code under `src/…` but does **not** include the `node-control-panel` repository source tree. I can therefore provide a rigorous integration interface plan for the control panel (based on the Node Technical Spec), but I cannot do a file-by-file inventory of that app **unless you provide a code snapshot** (e.g., zip of the control panel repository). This plan assumes you prefer to re-use the existing P2P port(s) for DAG messages (multiplexed message types) unless you explicitly want a physically separate DAG gossip port. I assume the DAG layer is initially for high-throughput dissemination and certified availability; deterministic ordering cuts are added only once availability is stable (this mirrors Synergy's own DAG plan phasing in `Synergy_DAG_Validator_Cluster_Integration_Plan.docx` ¶47–86).

Codebase inventory and current validator-cluster flow

Inventory of `synergy-testnet-beta/src` code snapshot

The uploaded `src.zip` expands to a Rust workspace with core modules relevant to consensus, P2P, RPC, validators/clusters, crypto, sync, and multiple role-bound binaries.

Core files (top-level):

Consensus: `src/consensus/consensus_algorithm.rs`, `dual_quorum.rs`, `vrf.rs`, `cartel_detection.rs`, `dao_governance.rs`, `synergy_score.rs`, `tests.rs`, `mod.rs` P2P: `src/p2p/messages.rs`, `networking.rs`, `mod.rs` RPC: `src/rpc/rpc_server.rs`, `src/rpc/mod.rs` Validator/clusters: `src/validator.rs` Block format: `src/block.rs` Config: `src/config/mod.rs` Crypto: `src/crypto/pqc.rs` Role runtime: `src/role_runtime.rs`, `src/role_profiles.rs` Addressing: `src/address.rs` and `src/synergy-address-engine/*`

File tree (from `src.zip`, max depth 2) is:

`src/Cargo.toml`
`src/address.rs`

`src/block.rs`
`src/config/mod.rs`
`src/consensus/*` (cartel_detection, consensus_algorithm, dao_governance, dual_quorum, synergy_score, tests, vrf)
`src/crypto/*`
`src/p2p/*`
`src/rpc/*`
`src/validator.rs`
`src/role_runtime.rs`, `src/role_profiles.rs`
`src/sync/*`
`src/bin/*` (role-specific node binaries)

Current flow as implemented

Transaction admission: P2P receives a `NetworkMessage::Transaction` and adds it to the shared transaction pool `TX_POOL` if not already present (`src/p2p/messages.rs` L20–22; `src/p2p/networking.rs` L1241–1269). RPC exposes many node monitoring methods for the control panel, including `synergy_getNodeStatus`, which returns peer count (via P2P), average block time, and sync status (`src/rpc/rpc_server.rs` L1197–1238).

Block proposal and finality: `ProofOfSynergy::execute()` runs a periodic loop. Each tick: it reads `TX_POOL` (`src/consensus/consensus_algorithm.rs` L315–318), selects a leader (`L358–367`), validates transactions (`L383–396`), creates a block proposal (`L405–414`), then runs dual-quorum consensus and commits the block on success. Finality uses dual thresholds: a weighted "validation quorum" (default 0.67) and a count-based "cooperation quorum" (default 0.51), then mints a QC (`src/consensus/dual_quorum.rs` L49–76, L182–198).

Cluster concepts exist, but are not yet consistently wired: Config includes `consensus.validator_cluster_size` (`src/config/mod.rs` L52–66, L176–193). Consensus retains `cluster_size` but currently uses it mostly for reward distro logic; it defaults to `30` if no config is loaded (`src/consensus/consensus_algorithm.rs` L200–204). Validator registry reorganizes clusters with a hard-coded maximum cluster size of `5` (`src/validator.rs` L313–316) and assigns cluster IDs/addressing (`L370–399).`
`The PoSy updated spec expects larger clusters (10–50, typical` C_target `~30) and epoch reshuffling (` PoSy_updated.docx` ¶91, ¶255–260). This mismatch must be fixed before a cluster-local DAG can be safe and deterministic.

Cryptographic and naming constraints: Canonical naming/encoding standards mandate Bech32m addresses derived from FN-DSA-1024 keypairs (`Synergy_Protocol_Naming_Encoding_Standards.docx` ¶47–48, ¶61, ¶148–151). The current implementation explicitly uses a hex placeholder instead of Bech32m (`src/address.rs` L10–19). This discrepancy affects identity and messaging formats—especially if DAG object identifiers are exposed over RPC and used in evidence bundles.

Deployment constraints: Your ports/nginx manual states "P2P ports: DNS-only, never proxied," "Metrics: localhost only," and defines an inter-node firewall policy including ranges for P2P and discovery (`Master`

`Network, Ports, and Nginx Configuration Manual.docx` ¶60–62, ¶852–854). DAG traffic that is validator-to-validator should remain within these P2P constraints.

Cluster-local certified DAG sublayer architecture

Why cluster-local certified DAG is the “right” fit for PoSy

Synergy already treats clustering and rotation as a security primitive and bounds the per-round consensus surface area (`03-proof-of-synergy-consensus.md`, cooperative clustering sections). A DAG layer must respect that: it should be scoped to the same cluster membership boundaries, not become a global “vertex gossip mesh.” This is explicitly called out in Synergy’s existing DAG integration plan decisions: “DAG scope is cluster-local,” and cross-cluster traffic should carry checkpoints/proofs, not unconstrained vertex gossip (`Synergy_DAG_Validator_Cluster_Integration_Plan.docx` ¶20–22).

In public literature, Narwhal and Tusk provide strong support for this separation-of-concerns approach: “separating the task of reliable transaction dissemination from transaction ordering” and using a DAG-based mempool for high-throughput dissemination. [1] This is directly compatible with PoSy: the DAG layer can be the dissemination/availability plane inside a cluster, while PoSy dual quorum remains the canonical finality plane.

Core DAG objects to implement

Synergy’s existing DAG plan already enumerates the minimum protocol objects needed for a real certified DAG layer (not just “gossip batches”) (`Synergy_DAG_Validator_Cluster_Integration_Plan.docx` ¶37–45). The key objects to make concrete in code are:

DagVertex: signed, deterministic envelope with `epoch_id`, `cluster_id`, `round`, `author_validator_id`, parents, and a commitment to the batch (root or blob reference). (Plan doc ¶38.) AvailabilityReceipt: signed ack stating the receiver actually has fetched/persisted the batch data. (Plan doc ¶39, plus “receipt without availability is worse than none,” ¶53–54.) DagCertificate: aggregated evidence that a threshold of distinct validators attested availability. (Plan doc ¶40, ¶54–55.) ClusterCheckpoint: compact export artifact binding: latest QC, DAG watermark/certificate root, and minimal cross-cluster summary material. (Plan doc ¶42, ¶69–74.)

These are also consistent with PoSy updated spec expectations around authenticated intra-/inter-cluster communications, replay protection, and bridge relay rate limits. For example: Intra-cluster is “direct peer-to-peer with ML-DSA authentication” (`PoSy_updated.docx` ¶96). Inter-cluster messages have timestamps and a 5-second validity window and sequence numbers to prevent replay (`PoSy_updated.docx` ¶279). Bridge relays are rate limited and produce a verifiable signature chain (`PoSy_updated.docx` ¶101).

Data flow and control flow diagrams

Cluster-local DAG + PoSy finality (high-level)

```
flowchart LR
  subgraph ClientPlane[Client / Wallet / SDK]
```

```
    C[Client] -->|JSON-RPC| RPC[RPC Gateway / Node RPC]
  end

  subgraph ValidatorNode[Validator Node]
    RPC -->|admission rules| MP[Mempool intake]
    MP --> DAG[DAG Engine<br/>(vertex builder + availability certs)]
    DAG -->|certified cut / checkpoint| PROP[PoSy Block Proposer]
    PROP --> DQ[Dual-Quorum Voting + QC]
    DQ --> CHAIN[(Chain commit)]
    CHAIN -->|block broadcast| P2P[P2P Gossip]
  end

  subgraph ClusterPeers[Cluster Peers]
    DAG <-->|vertex + receipt + fetch| P2P
    DQ <-->|votes + QC| P2P
  end

  CHAIN --> Aud[Audit/Archive nodes]
```

Within-cluster certified DAG lifecycle (simplified)

```
sequenceDiagram
  participant V as Validator (author)
  participant P as Cluster Peer
  participant D as DAG Store

  V->>D: build DagVertex(round r, parents, batch_root)
  V->>P: gossip VertexEnvelope(vertex, sig)
  P->>D: fetch batch if missing + persist
  P->>V: AvailabilityReceipt(vertex_id, have_blob=true, sig)
  V->>D: aggregate receipts → DagCertificate(2f+1)
  V->>P: broadcast DagCertificate
```

Eight-phase integration plan with code-level touchpoints

The phases below collapse Synergy's internal draft "Phase 0–8" plan into **eight phases** by merging "cross-cluster checkpointing" and "execution DAG" sequencing into one structured rollout (availability-first), while keeping all key deliverables and tying them to the actual code touchpoints in `src.zip`.

Phase foundation and parameter canonicalization

Goal: Remove specification drift so DAG invariants are well-defined and deterministic across nodes.

Lock these decisions as a short ADR (as already recommended) (`Synergy_DAG_Validator_Cluster_Integration_Plan.docx` ¶29): Finality remains PoSy QC-based

(dual quorum). DAG is cluster-local; cross-cluster traffic = checkpoints and compact proofs only. Availability certification is mandatory before ordering. Synergy Score weighting belongs to PoSy finality, not to “vertex existence.”

Concrete fixes required in code/spec alignment:

Unify cluster sizing: PoSy updated: clusters 10–50 (`PoSy_updated.docx` ¶91) with typical `C_target ~30` (`¶257`). Rust validator registry: `MAX_CLUSTER_SIZE = 5` (`src/validator.rs` L313–316). Config default: `validator_cluster_size = 7` (`src/config/mod.rs` L176–183) vs consensus fallback to `30` (`src/consensus/consensus_algorithm.rs` L200–204). Action: Introduce a single canonical `C_target` source-of-truth (config + governance parameter) and rework `ValidatorRegistry::reorganize_clusters()` and/or `ValidatorRotation` usage so every node computes the same cluster membership at epoch boundaries (matching PoSy updated epoch reshuffling logic, `PoSy_updated.docx` ¶255–260).

Standardize cryptographic schemes and message constraints: PoSy updated specifies ML-DSA signatures for votes and proposals (`PoSy_updated.docx` ¶196, ¶214), while the current Rust code signs blocks/votes via PQC manager using FN-DSA (`src/consensus/consensus_algorithm.rs` L105–116; `src/consensus/dual_quorum.rs` L142–155). Action: Decide whether Testnet-Beta DAG objects use FN-DSA (latency-sensitive) or ML-DSA (spec default) and document it. The naming standard explicitly positions FN-DSA for latency-sensitive operations (`Synergy_Protocol_Naming_Encoding_Standards.docx` ¶148–151).

Phase protocol objects and storage modules

Goal: Make DAG objects real, signed, storable, auditable artifacts (not “just messages”).

Add new Rust modules (new files): `src/dag/mod.rs` `src/dag/types.rs` (DagVertex, AvailabilityReceipt, DagCertificate, ClusterCheckpoint) `src/dag/store.rs` (separate from canonical chain store; recommended for rollback and audit separation) (`Synergy_DAG_Validator_Cluster_Integration_Plan.docx` ¶55–56) `src/dag/certs.rs` (threshold policies, signer bitmap, aggregation) `src/dag/order.rs` (deterministic ordering cut selection; keep minimal at first)

Decide identifier encodings: DAG object IDs must be stable, typed, and safe for APIs. Naming standards require Bech32m for addresses and lowercase hex (no base64) for signature serialization (`Synergy_Protocol_Naming_Encoding_Standards.docx` ¶47–48, ¶61, ¶185). Current code sometimes uses Base64 decoding for wallet public keys (`src/consensus/consensus_algorithm.rs` near `get_transaction_public_key`), which is explicitly incompatible with “never Base64 for signatures” rule (Naming standard ¶185). Fix prior to DAG RPC.

Phase cluster-local DAG availability engine implementation

Goal: Implement the certified DAG dissemination plane in each cluster; do not implement global ordering yet.

Core invariants to implement (from Synergy's existing plan): One-author-one-vertex-per-round (beta simplification) to make equivocation detection trivial (`Synergy_DAG_Validator_Cluster_Integration_Plan.docx` ¶51). Parent selection requires diversity; avoid clique parent sets (`¶52`). Peers must fetch/persist batch before acknowledging (`¶53–54`). Certificates only when threshold met, typically 2f+1 (`¶54–55`).

Code-level touchpoints:

Replace direct `TX_POOL` consumption with DAG ingestion: P2P currently inserts transactions into `TX_POOL` (`src/p2p/networking.rs` L1262–1269). Consensus currently clones `TX_POOL` for proposals (`src/consensus/consensus_algorithm.rs` L315–397). Action: Introduce a new shared `DAG_INGRESS` channel or `Arc<Mutex<DagEngine>>`. P2P should push transactions into DAG ingress; consensus should request a certified checkpoint/cut from the DAG engine.

Add cluster scoping: P2P handshake includes an optional `validator_address` field (`src/p2p/messages.rs` L8–16), but `PeerConnection` does not currently persist it (it only sets `node_id`, `version`, capabilities, public address; `src/p2p/networking.rs` L1042–1049). Action: Extend `PeerConnection` to store `validator_address: Option<String>` and derive `cluster_id` from `ValidatorManager` so DAG messages can be sent only to cluster peers.

Phase P2P integration and message handlers

Goal: Make DAG dissemination real on the wire, without breaking existing block/tx propagation.

Add new message types in `src/p2p/messages.rs`: `DagVertex { vertex: DagVertexEnvelope }` `DagReceipt { receipt: AvailabilityReceipt }` `DagCertificate { cert: DagCertificate }` `DagFetchBatch { vertex_id }` / `DagBatch { vertex_id, blob }` `DagFetchVertex { vertex_id }` (missing parent path)

Update `handle_messages()` in `src/p2p/networking.rs`: Currently a match handles `Handshake`, `Block`, `Transaction`, etc (`src/p2p/networking.rs` L995+). You will add cases to: Validate signatures and replay constraints (timestamp windows / sequence numbers per session; see PoSy updated replay rules, `PoSy_updated.docx` ¶279). Enforce cluster-only acceptance: reject DAG messages from peers not in the same cluster (or queue for audit evidence if you want evidence of attempted abuse). Apply rate limiting per public key / validator address, modeled on the bridge relay limits described in PoSy updated (`PoSy_updated.docx` ¶101).

Phase PoSy consensus integration and checkpoint binding

Goal: Make PoSy propose blocks from DAG-certified availability rather than raw mempool, while retaining dual-quorum finality.

Critical integration change: Today, PoSy's proposer does: `let pool = TX_POOL.lock(); ... processed_transactions = validate(tx); create_block_proposal(processed_transactions)` (`src/consensus/consensus_algorithm.rs` L315–414). With DAG: PoSy proposer should instead request `DagCut` / `ClusterCheckpoint`: Wait for a

certified cut (bounded wait; never block indefinitely). Pull tx payloads from local DAG store (already guaranteed available by receipts). Build the block from that cut's tx list and also record a commitment (e.g., `dag_cut_root`).

Block format changes: Block hash currently commits to `transactions_root` only (`src/block.rs` L52–78). If DAG certificates become security-critical evidence, you must bind the DAG cut/certificate into the block header hash. Minimal approach: Add `dag_cut_root` (string) or `dag_certificate_root`. Compute block hash over `{block_index, prev_hash, validator_id, nonce, timestamp, transactions_root, dag_cut_root}`. Update validation logic in `DualQuorumConsensus::is_block_hash_valid()` accordingly (`src/consensus/dual_quorum.rs` L99–116).

Dual quorum checkpoint verification: Extend `DualQuorumConsensus::validate_block_proposal()` to validate: DAG cut certificate is well-formed and meets threshold (2f+1) for the current cluster membership. No equivocation evidence exists for included vertices (or else proposer is faulty). All referenced blobs are locally available.

Pseudocode sketch: modify PoSy propose loop

```
// BEFORE: clone TX_POOL → validate txs → propose block
let txs = TX_POOL.lock().unwrap().clone();

// AFTER: request certified DAG cut
let cut = dag_engine.next_certified_cut(epoch, cluster_id, max_txs, timeout)?;
let txs = dag_engine.materialize_transactions(&cut)?;
let new_block = create_block_proposal_with_dag_root(prev, leader, txs,
cut.root(), pqc);
```

Phase exit criteria: Blocks are produced from certified DAG cuts. Validators reject blocks whose DAG roots/certs fail verification (fail-closed). No uncertified vertex influences ordering (non-negotiable rule, `Synergy_DAG_Validator_Cluster_Integration_Plan.docx` ¶118).

Phase Cross-cluster checkpoints and epoch transitions

Goal: Preserve Synergy's "clusters + bridge relays" worldview. No global DAG gossip.

PoSy updated describes bridge relays, signature chaining, rate limiting, and redundancy requirements (`PoSy_updated.docx` ¶101). The DAG plan similarly insists cross-cluster traffic carry compact checkpoint packages, not full vertex graphs (`Synergy_DAG_Validator_Cluster_Integration_Plan.docx` ¶69–74).

Implementation steps: Define `ClusterCheckpoint` as the only outbound cross-cluster DAG artifact: latest QC hash, DAG watermark, certificate root, and any light-client relevant summary. Integrate checkpoint exchange into existing P2P or SXCP relay plane (depending on which plane owns cross-cluster protocols; the plan doc treats it as an authenticated relay concept). Epoch boundary hard rule: old epoch

vertices expire; nothing can silently leak forward (`Synergy_DAG_Validator_Cluster_Integration_Plan.docx` ¶72–74). This aligns with PoSy updated epoch boundary behavior and frequent rotation design (`PoSy_updated.docx` ¶244–260, ¶375).

Phase RPC extensions and control plane integration

Goal: Make DAG observable, auditable, and safe to operate—without making the control panel a privileged attack surface.

RPC additions: Your existing `rpc-methods.md` enumerates the current JSON-RPC surface and already includes node monitoring methods for the control panel (e.g., `synergy_getNodeStatus`) (`rpc-methods.md` L1–40 and beyond; plus implementation at `src/rpc/rpc_server.rs` L1197–1238). Synergy's DAG plan recommends RPC methods such as: `synergy_getDagVertex`, `synergy_getDagCertificate`, `synergy_getDagEquivocations`, `synergy_getDagAvailabilityLag` (`Synergy_DAG_Validator_Cluster_Integration_Plan.docx` ¶143–152).

Implementation: Extend `src/rpc/rpc_server.rs` `handle_json_rpc()` match to include the DAG calls. Extend `synergy_getNodeStatus` object schema to include DAG health metrics (frontier depth, highest certified round, missing-parent rate, cert latency). This is explicitly recommended (`Synergy_DAG_Validator_Cluster_Integration_Plan.docx` ¶98).

Node Control Panel integration (interface plan; code not provided): The Node Technical Spec says the Control Panel should expose consensus/profile configuration as a typed overlay and produce `node.toml` and role overlays rather than making operators hand-edit schema (`Synergy_Network_Node_Technical_Specification.docx` ¶159–164, ¶201, ¶213–221). It also emphasizes quarantine/degraded states as first-class lifecycle outcomes (`¶173–174, ¶191, and table hits around "Quarantined").`
`Therefore, the control panel changes should be:`
`Add DAG-aware toggles + safe defaults (e.g.,` quarantine_on_dag_integrity_failure = true `as suggested in the DAG plan config appendix,` Synergy_DAG_Validator_Cluster_Integration_Plan.docx` ¶163–165). Display DAG health and evidence availability via new RPC calls; treat suspicious conditions as "degraded/quarantined" states—not "warning only."

To produce a real file-by-file patch plan for the control panel repo, you'll need to provide the repository snapshot (zip or exported file tree). Right now, only the spec-level control panel architecture is accessible.

Phase Rollout, deployment hardening, and operator readiness

Goal: Ship in controlled blast-radius stages with strong rollback.

Rollout staging (already recommended): Stage 1: single-cluster internal run Stage 2: two clusters + bridge relays Stage 3: adversarial simulation suite: equivocation, hidden subgraphs, withholding, spam bursts, partitions, stale/replay (`Synergy_DAG_Validator_Cluster_Integration_Plan.docx` ¶107–109).

Deployment constraints: Do not proxy P2P/DAG gossip ports via nginx (“P2P ports: DNS-only, never proxied”) (`Master Network, Ports, and Nginx Configuration Manual.docx` ¶60). Keep metrics localhost-only (`¶62, ¶854`). If you introduce any new inter-node port(s) for DAG traffic, they must remain in the inter-node allowlist policy (e.g., within P2P ranges `38638–38652` depending on environment) (`Master Network, Ports, and Nginx Configuration Manual.docx` ¶852–853).

File-by-file change table for `synergy-testnet-beta/src`

The table below is intentionally “file-by-file” and concise (short phrases only), with new modules included.

File / module	Change type	Summary
`src/dag/mod.rs`	New	DAG engine wiring + public entrypoints
`src/dag/types.rs`	New	`DagVertex`, `AvailabilityReceipt`, `DagCertificate`, `ClusterCheckpoint`
`src/dag/store.rs`	New	Separate DAG persistence + GC/pruning
`src/dag/certs.rs`	New	Threshold policy, signer bitmap, cert validation
`src/dag/order.rs`	New	Deterministic cut selection (availability-first)
`src/consensus/consensus_algorithm.rs`	Modify	Replace `TX_POOL` intake with `DagEngine` cut intake; bind DAG root into proposal
`src/consensus/dual_quorum.rs`	Modify	Validate DAG roots/certs in `validate_block_proposal`; ensure block hash includes DAG commitment
`src/block.rs`	Modify	Add `dag_cut_root` / `dag_checkpoint_root`; include in block hash + header
`src/p2p/messages.rs`	Modify	Add DAG message variants (vertex/receipt/cert/fetch)
`src/p2p/networking.rs`	Modify	Cluster-scoped DAG gossip; rate limiting; replay checks; store peer validator identity
`src/rpc/rpc_server.rs`	Modify	Add DAG RPCs + DAG health fields in `synergy_getNodeStatus`
`src/config/mod.rs`	Modify	Add `[dag]` config keys (thresholds, batching, timeouts, evidence)
`src/validator.rs`	Modify	Canonicalize cluster sizing + epoch reshuffle logic; add DAG slashing reasons
`src/consensus/cartel_detection.rs`	Modify	Extend to DAG-specific collusion signals; evidence hooks

File / module	Change type	Summary
`src/consensus/dao_governance.rs`	Modify	Add emergency parameter toggles for DAG shutdown/quarantine
`src/role_runtime.rs`	Modify	Start DAG engine under validator/committee profiles; expose status in runtime report
`src/address.rs`	Modify	Align with Bech32m requirement (remove hex placeholder)

DAG security threat matrix and enforcement model

Security principles to treat as non-negotiable

The DAG plan calls out "No uncertified ordering" as foundational (`Synergy_DAG_Validator_Cluster_Integration_Plan.docx` ¶118). This becomes a consensus-layer invariant when DAG roots/certs are bound into blocks: uncertified vertices "do not exist" for ordering.

Synergy's security implementation spec emphasizes that consensus-layer rules must be enforced before execution and fail closed (`Synergy_Network_Security_Implementation_Specification.docx` ¶22). It also highlights replay risks and mandates context binding for signatures (`¶65–69). PoSy updated similarly mandates timestamps and sequence numbers with narrow validity windows for inter-cluster messages (` PoSy_updated.docx` ¶279). These become direct requirements for DAG messages (vertex/receipt/cert) and for DAG fetch.

Threat matrix with mitigations, detection hooks, and enforcement actions

DAG attack	Typical mechanism	Detection hooks to add	Mitigations	Enforcement actions
Equivocation	Same author emits 2 vertices for same round	`one-author-one-vertex-per-round` invariant; emit signed equivocation proof	Reject equivocation vertex; certificate requires unique authors	Slash reason `dag_equivocation_same_rou` reassignment (`src/validator.rs` L411–42

DAG attack	Typical mechanism	Detection hooks to add	Mitigations	Enforcement actions
Certificate fraud	Forged receipt/cert bitmap	Verify PQ signatures and signer membership; bitmap correctness	Cert verification before ordering; deterministic signer ordering	Slash `dag_certificate_fraud`; emergency widespread
Data withholding	Author proposes vertex but peers can't fetch blob	Receipt requires "have blob"; track "certified-but-unavailable" events	Require fetch-before-receipt; store-level verification	Escalating penalty: Synergy Score decay + jail; audit (`Synergy_DAG_Validator_Cluster_Integ` ¶91–92)
Spam / DoS	Flood vertices/ receipts/ fetches	Per-peer rate limit + per-validator quotas; size caps	Cluster-only acceptance; bounded batch sizes	Throttle + temporary ban; persistent offenders
Eclipse / partition	Attacker isolates node so it sees biased DAG	Peer diversity metrics; detect abnormal peer set, missing parents	Strict allowlist mode (`src/config/mod.rs` L120–132); multi-bootnode policy	Quarantine node (control panel lifecycle); refu
Sybil	Many fake identities join cluster	Require validator identity binding; disable auto-register in production	Strict allowlist; stake + governance admission	Reject DAG messages from non-validators; do sets
Replay / stale injection	Re-send old vertices/certs	Enforce timestamp/ sequence windows	Include chain_id/epoch/ round in signed data; 5s window (`PoSy_updated.docx` ¶279)	Drop + strike count; jail if repeated malicious

DAG attack	Typical mechanism	Detection hooks to add	Mitigations	Enforcement actions
Censorship patterns	Leader excludes vertices systematically	Compare certified frontier vs ordered cut; audit "missing eligible certs"	Deterministic cut policy + proposer accountability	Slashing tag `dag_censorship_pattern`; go remove proposer privileges

How this ties into existing Synergy modules:

Validator penalties: `ValidatorRegistry::slash_validator()` currently supports only `"double_sign"` and `"inactivity"` (`src/validator.rs` L411–424). DAG enforcement requires adding explicit slashing reasons (as already recommended): `dag_equivocation_same_round`, `dag_certificate_fraud`, `dag_data_withholding`, `dag_censorship_pattern`, `dag_replay_or_stale_spam` (`Synergy_DAG_Validator_Cluster_Integration_Plan.docx` ¶166–172).

Cartel/collusion detection: `CartelDetectionEngine` is currently vote-correlation oriented (`src/consensus/cartel_detection.rs`), but the same approach can consume DAG evidence: correlated withholding, correlated denial of receipts, correlated censorship. Evidence bundles should be produced and served via DAG RPC for audit nodes (`Synergy_DAG_Validator_Cluster_Integration_Plan.docx` ¶91–92).

Governance and emergency actions: The DAO framework supports emergency actions (`src/consensus/dao_governance.rs` defines `ProposalType::EmergencyAction`). You should add parameter toggles to: (a) disable DAG ordering while preserving PoSy block production; (b) raise thresholds; (c) quarantine clusters. This matches PoSy updated governance parameterization of cluster size/quorum thresholds (`PoSy_updated.docx` ¶306+) and Synergy's "fail into quarantine" posture (`Synergy_Network_Node_Technical_Specification.docx` ¶173–174, ¶191).

Tests, benchmarks, release gates, milestones, and rollback

Required tests and benchmarks

DAG invariants test suite (core correctness): Property tests: acyclicity, uniqueness of author/round, certificate validity, deterministic ordering, epoch-boundary expiry. This is explicitly recommended (`Synergy_DAG_Validator_Cluster_Integration_Plan.docx` ¶125–126). Cross-validation: every node computing same ordering cut from same certified frontier.

Adversarial simulation: Equivocation, hidden subgraph attempts, withholding, spam bursts, partitions, bridge loss, stale/replay injections (`Synergy_DAG_Validator_Cluster_Integration_Plan.docx` ¶108–109).

Performance benchmarks: In-cluster throughput graph: tx/sec vs cluster size. Certificate latency distribution: vertex→cert time, cert→cut time. CPU impact of PQ signature verification (choose FN-DSA vs ML-DSA policy consistent with naming standard guidance on latency (`Synergy_Protocol_Naming_Encoding_Standards.docx` ¶148–151)).

Release gates

Security gates (hard requirements): Security spec mandates automated unit tests for enforcement behaviors and a CI gate that does not allow reduced pass rates (`Synergy_Network_Security_Implementation_Specification.docx` ¶188). Security spec also demands third-party security audit coverage for consensus-layer enforcement modules before testnet launch, and no unresolved Critical/High findings (`Synergy_Network_Security_Implementation_Specification.docx` ¶194). The DAG availability layer must be explicitly added to audit scope (as the DAG plan also recommends, ¶129).

Operational gates: No DAG traffic is exposed via nginx; P2P is never proxied (`Master Network, Ports, and Nginx Configuration Manual.docx` ¶60). Metrics remain localhost-only (`¶62, ¶854`). Node Control Panel must display "degraded/quarantined" states and integrate DAG health without adding privileged controls (`Synergy_Network_Node_Technical_Specification.docx` ¶173–174, ¶191).

30/60/90-day milestones

Day 30: Certified DAG inside one cluster (availability-first) Canonicalize cluster sizing and epoch rotation parameters across config + registry + consensus (fix mismatch between `src/validator.rs` max=5 and PoSy updated 10–50). Implement DAG object types + signatures + storage + basic gossip. Implement receipts + 2f+1 certificate formation. Add basic DAG RPC calls (read-only) and DAG health fields in `synergy_getNodeStatus` (`src/rpc/rpc_server.rs` control panel integration point at L1197–1238).

Day 60: PoSy block proposal from DAG + consensus binding Replace TX_POOL consumption in `ProofOfSynergy::execute()` with certified DAG cut selection (`src/consensus/consensus_algorithm.rs` L315–414 touchpoint). Bind DAG cut root into block hash; update block validation (`src/block.rs`, `src/consensus/dual_quorum.rs`). Add equivocation proofs and enforce slashing hooks (extend `src/validator.rs` L411–424). Implement "Stage 1" and "Stage 2" rollout: single cluster then two clusters + checkpoint exchange (`Synergy_DAG_Validator_Cluster_Integration_Plan.docx` ¶107–108).

Day 90: Cross-cluster checkpointing, bridges, full adversarial simulation, rollback readiness Implement cluster checkpoint objects and bridge relay path with rate limiting and signature chaining consistent with PoSy updated (`PoSy_updated.docx` ¶101, ¶279). Add evidence bundles for audit reconstruction (`Synergy_DAG_Validator_Cluster_Integration_Plan.docx` ¶91–92). Run full adversarial suite; set thresholds and emergency toggles in DAO.

Rollback plan

A credible rollback must preserve chain safety and operator clarity:

Soft rollback (preferred): “DAG disabled, PoSy continues” Add a governance-controlled flag `dag.enabled=false` that switches proposer from DAG cut back to TX_POOL, while leaving all DAG storage intact for audit. This preserves block production (PoSy finality unaffected) and avoids chain halts. Control panel should show “DAG disabled by governance” state; nodes remain “healthy” if PoSy is finalizing.

Hard rollback: “quarantine cluster / stop finality artifacts” If a cluster’s DAG integrity is compromised (certificate fraud, persistent withholding), validators in that cluster should enter quarantine mode (consistent with Node Technical Spec’s “do nothing unsafe, loudly” model) (`Synergy_Network_Node_Technical_Specification.docx` ¶173–174, ¶191). In quarantine, the node stops emitting authoritative DAG/QC artifacts and requires operator intervention or governance action.

Evidence preservation: DAG store is separated from chain store so rollback does not erase audit trails (`Synergy_DAG_Validator_Cluster_Integration_Plan.docx` ¶55–56, ¶91–92).

What’s missing from this report and what to provide to close the gap

This report fully inventories and maps touchpoints for `synergy-testnet-beta/src` from the supplied `src.zip`, and it grounds the DAG design in the PoSy specs, security specs, and canonical DAG-BFT literature. [3]

The only major missing component is a file-tree and patch plan for the `synergy-node-control-panel` repository source. The Node Technical Spec provides a strong architectural direction for what the control panel must do (typed overlays, quarantine states, safe key handling) (`Synergy_Network_Node_Technical_Specification.docx` ¶159–164, ¶173–174, ¶191, ¶201), but a **file-by-file** table for that repo requires the actual code snapshot (zip or exported tree).

---

[1] [3] Narwhal and Tusk: A DAG-based mempool and efficient BFT consensus
https://research-explorer.ista.ac.at/record/11331

[2] [2201.05677] Bullshark: DAG BFT Protocols Made Practical
https://arxiv.org/abs/2201.05677